

Blokus v2: A Bitboard Engine and a Budgeted Bandit-Search AI

Technical Report

Feng-Ting Liao

July 2026

Abstract

We describe the engine and artificial intelligence of *Blokus v2*, a Rust implementation of the four-player board game Blokus. The rules engine encodes the official 20×20 board as per-player bitboards, reducing legality testing and full legal-move enumeration to word-parallel bit operations (≈ 0.2 ms per move including evaluation). On top of the engine we build two policies: (i) a fast heuristic policy that scores every legal move on piece size, corner mobility, opponent-corner denial, and early-game centrality; and (ii) a strong policy that treats move selection as a budgeted multi-armed bandit problem, allocating Monte-Carlo rollouts across the top- K heuristic candidates with UCB1 and returning the most-visited arm when a wall-clock budget expires. In head-to-head evaluation the bandit policy defeats the heuristic policy in 11 of 12 four-player games while honoring a one-second decision budget to within a single iteration. The system exposes the budget to players directly as a “think-time” dial (0–5 s) in the game’s interface.

1 Introduction

Blokus is a four-player perfect-information territory game played with polyomino pieces. Its branching factor is unusually punishing: an opening position offers each player on the order of 10^3 legal placements, and placements interact through both denial (occupying cells opponents need) and mobility (creating one’s own future placement anchors). These properties make Blokus a pleasant middle ground between classic two-player search domains and modern multi-agent settings: strong play requires lookahead, but the four-player, single-move-per-turn structure defeats naive minimax.

This report documents the second major version of the project. Version 1 was a C++ course project at National Chiao Tung University (now National Yang Ming Chiao Tung University) by Feng-Ting Liao and ShengHsiung Hsieh; its AI scanned a 14×14 window around a single frontier cell and placed near-randomly in the opening. Version 2 is a ground-up Rust rewrite whose AI is the subject of this report. We cover the board representation (Section 3), the heuristic evaluation (Section 4), the bandit search (Section 5), and empirical results (Section 6).

2 Rules and Notation

The board is the grid $B = \{0, \dots, 19\}^2$. Each player $p \in \{0, 1, 2, 3\}$ owns the 21 free polyominoes of sizes 1–5 (89 squares total) and a starting corner c_p (one of the four board corners). Players

move in fixed rotation; a player unable to move is permanently eliminated from the rotation (opponent pieces can only remove placement options, never create them, so inability to move is absorbing).

A *placement* is a piece, one of its up to eight orientations (rotations and reflections, deduplicated per piece), and a translation. Let $S \subset B$ be the placement’s cells and O_p the cells already occupied by p . A placement for p is legal iff

1. S lies within B and is disjoint from *all* occupied cells;
2. S is not 4-adjacent (edge-adjacent) to O_p ;
3. S is 8-but-not-4 adjacent (corner-adjacent) to O_p ; or, if $O_p = \emptyset$, $c_p \in S$.

Contact with *other* colors is unrestricted. Terminal scoring is official: -1 per unplaced square, $+15$ if all 21 pieces were placed, and $+5$ more if the monomino was placed last.

3 Bitboard Engine

3.1 Representation

Each player’s occupancy O_p is stored as twenty 32-bit words, one per row, using the low 20 bits (≈ 80 bytes per player). The two neighborhood operators used throughout are computed by shifts and ORs over rows:

$$\begin{aligned} \text{orth}(X) &= X \cup \{\text{cells 4-adjacent to } X\}, \\ \text{diag}(X) &= \{\text{cells 8-adjacent but not necessarily 4-adjacent to } X\} \text{ (diagonal spread)}. \end{aligned}$$

Both are 20-iteration loops of word operations; no per-cell work is done.

3.2 Forbidden cells and seeds

For player p define

$$F_p = \left(\bigcup_q O_q \right) \cup \text{orth}(O_p) \quad (\text{forbidden: cells no piece of } p \text{ may cover}), \quad (1)$$

$$A_p = \text{diag}(O_p) \setminus F_p \quad (\text{seeds: legal anchor cells}), \quad (2)$$

with $A_p = \{c_p\}$ before p ’s first move. Legality of a placement S reduces to $S \cap F_p = \emptyset$ and $S \cap A_p \neq \emptyset$: rule (1) and (2) are exactly $S \cap F_p = \emptyset$, and rule (3) is $S \cap A_p \neq \emptyset$ because any placement satisfying (1)–(2) touches O_p diagonally iff it covers a seed.

3.3 Move generation

Legal moves are enumerated by aligning, for every unused piece, every orientation cell onto every seed and testing the (at most five) cells of the placement against F_p ; duplicates arising from a placement covering several seeds are removed with a hash set keyed on (piece, orientation, offset). The count of seeds is small (typically 5–30), so enumeration touches only a tiny fraction of the $20 \times 20 \times 8 \times 21$ nominal placement space. $|A_p|$ (`seed_count`) doubles as a cheap mobility measure for evaluation.

4 Heuristic Policy

The fast policy scores every legal move m of player p with

$$V(m) = 90 \text{size}(m) + 14 \text{corners}(m) + 28 \text{blocked}(m) + w_c \text{center}(m), \quad (3)$$

where size is the piece’s square count (points saved under official scoring); corners is $|A_p|$ recomputed *after* the move (future mobility); blocked is the number of opponent seed cells covered by the move, summed over active opponents (each is one placement anchor an opponent loses); and center sums, over the move’s cells, $9 - d_\infty(\text{cell}, \text{center } 2 \times 2)$, with weight $w_c = 4$ during the first five rounds and 1 afterwards. The weights were tuned by self-play.

Near-tie sampling. Instead of a strict $\arg \max$, the policy samples uniformly among all moves with $V(m) \geq \max_{m'} V(m') - \tau$, $\tau = 25$. This costs no measurable strength and removes the deterministic “same opening every game” artifact (measured in Section 6.5).

5 Bandit Search

The strong policy treats the decision as a *budgeted best-arm identification* problem: given a wall-clock budget T (the in-game think dial, 0–5 s), spend rollouts where regret arguments say they are most informative, then commit to one move. Algorithm 1 gives the full procedure; constants are in Table 1.

5.1 Arms and priors

All legal moves are scored with Eq. (3) and the top $K = 40$ become bandit arms. Arm i with static score s_i is initialized with $n_0 = 2$ virtual visits at value

$$\hat{v}_i^{(0)} = 0.35 + 0.3 \frac{s_i - s_{\min}}{s_{\max} - s_{\min}} + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{U}(-0.01, 0.01), \quad (4)$$

so that early pulls follow the heuristic ordering without hard-committing to it; the jitter breaks exact ties among symmetric openings.

5.2 UCB1 allocation

At iteration t the search pulls the arm maximizing

$$\text{UCB}_i = \hat{v}_i + C \sqrt{\frac{\ln(t+1)}{n_i+1}}, \quad C = 0.45, \quad (5)$$

where $\hat{v}_i$ is the arm’s running mean value in $[0, 1]$ and n_i its visit count (including virtual visits). UCB1’s logarithmic-regret guarantee [1] is what makes the think-time dial behave sensibly: additional budget is spent almost exclusively on arms still statistically capable of being best.

Algorithm 1 Budgeted UCB1 move selection for player p

```
1:  $M \leftarrow \text{legal\_moves}(p)$ ; return the move if  $|M| \leq 1$ ; greedy policy if  $T = 0$ 
2: score  $M$  with Eq. (3); keep top  $K$  as arms with priors  $(n_0, \hat{v}_i^{(0)})$ 
3: while elapsed  $< T$  do
4:    $i \leftarrow \arg \max_i \hat{v}_i + C\sqrt{\ln(t+1)/(n_i+1)}$ 
5:    $G \leftarrow$  copy of root state; play  $m_i$ 
6:   for  $h = 1 \dots H$  while  $G$  not terminal do
7:     current player plays  $\arg \max_m V(m) + \sigma \text{Gumbel}()$   $\triangleright$  softmax rollout
8:   end for
9:   update arm  $i$  with leaf value  $v(G)$ ;  $t \leftarrow t + 1$ 
10: end while
11: return uniform choice among arms with  $n_i \geq 0.9 \max_j n_j$ 
```

5.3 Rollout policy

A pull of arm i plays m_i , then simulates $H = 8$ further plies (two full rounds) in which *every* player, friend and foe, follows a *noisy-greedy* policy: it plays $\arg \max_m [V(m) + \sigma G_m]$ with i.i.d. standard Gumbel noise G_m and scale $\sigma = 25$ evaluation units. By the Gumbel-max (perturb-and-max) identity this is exactly softmax sampling over V/σ [3, 4], giving rollouts principled variety at zero bookkeeping cost. Plies whose position offers more than 96 legal moves evaluate a uniform sample of 96 of them, capping rollout latency around 1 ms with no measured strength loss. Eliminated players are skipped by the engine’s turn logic.

5.4 Leaf evaluation

Rollouts are truncated, so leaves are scored by a potential function. For a terminal leaf the official score differential against the best opponent is used; otherwise

$$\phi(q) = \text{placed}(q) + 0.4 |A_q|, \quad v = \frac{1}{2} + \frac{1}{2} \tanh\left(\frac{\phi(p) - \max_{o \neq p, o \text{ active}} \phi(o)}{12}\right), \quad (6)$$

i.e., squares already banked plus discounted mobility, squashed to $[0, 1]$. Measuring against the *best* opponent rather than the average reflects that Blokus is won by rank, not by point sum.

5.5 Final selection

When the budget expires the search returns the most-visited arm, with one refinement: arms whose visit count is within 90% of the maximum are statistically indistinguishable after a few hundred rollouts, so the policy picks uniformly among them (exact ties fall back to mean value). This preserves strength while keeping openings varied between games.

6 Benchmarks

All measurements: Apple Silicon macOS, `cargo run --release (LTO, opt-level=3)`, single-threaded search. The benchmark driver is `examples/selfplay.rs`; the numbers below are from the run recorded at the time of writing and are reproduced (within noise) by `cargo run --release --example selfplay`.

Table 1: Search constants (all tuned by self-play).

Constant	Meaning	Value
K (TOP_K)	candidate arms	40
C (UCB_C)	UCB1 exploration	0.45
n_0 (PRIOR_N)	virtual prior visits per arm	2
H (ROLLOUT_PLIES)	rollout horizon (plies)	8
σ (GUMBEL_SCALE)	rollout softmax temperature	25
— (ROLLOUT_MOVE_CAP)	moves evaluated per rollout ply	96
τ (GREEDY_TOLERANCE)	greedy near-tie band	25

6.1 Engine and heuristic-policy speed

Twenty full four-player self-play games with the heuristic policy complete in ≈ 0.25 s total — **≈ 0.19 ms per move** including full legal-move enumeration and evaluation of every candidate. This is the latency floor of the “dial = 0” setting.

6.2 Heuristic policy vs. random baseline

Seats 0 and 2 play the heuristic policy; seats 1 and 3 play uniformly random legal moves (comparable to v1’s opening behavior). Over 20 games the heuristic side’s best score beats the random side’s best score **20/20** times, with mean official scores -8.7 vs. -37.3 .

6.3 Bandit search vs. heuristic policy

Seats 0 and 2 play the bandit search at 300 iterations (≈ 350 ms per decision); seats 1 and 3 play the heuristic policy. Over 12 games:

Table 2: Bandit (300 iterations) vs. heuristic policy, 12 four-player games.

	Bandit seats	Heuristic seats
Wins (best score in game)	11/12	1/12
Mean official score	-11.6	-20.0

An 8.4-point mean margin is roughly two pentominoes of extra territory per game; in one game the bandit seat placed all 21 pieces ($+20$, the maximum score with monomino-last bonus).

6.4 Budget compliance

With `Budget::Millis(1000)`, decisions from an opening position and a round-6 midgame position both returned in 1.001 s — the budget is checked before every rollout, so the worst-case overshoot is a single rollout (≈ 1 –5 ms). At 300 iterations the mean decision time over 430 decisions was 349 ms.

6.5 Opening diversity

Over ten RNG seeds, the number of distinct first moves played as Blue: v2’s original strict-argmax greedy **1/10** (the user-visible “the first pieces are always the same” defect); near-tie greedy

8/10; bandit search 5/10. The bandit’s diversity is lower than the greedy’s by design — rollouts genuinely disambiguate some openings the static evaluation cannot.

7 Discussion and Future Work

The design deliberately spends its budget at the root rather than growing a tree: with four players and per-ply branching in the hundreds, a UCT tree [2] of meaningful depth is unreachable in a few seconds, whereas root-level bandit allocation with strong heuristic rollouts converts budget into strength immediately and degrades gracefully to the heuristic policy at $T = 0$. Natural extensions, roughly in order of expected value per effort:

- **Progressive widening / tree growth** below the root for the late game, where branching collapses and exact lines matter;
- **Paranoid or $\max^n$ backups** in place of the best-opponent potential, sharpening play when one opponent is far ahead;
- **Learned evaluation** replacing Eq. (3)’s hand-tuned weights (the bitboard features are already network-friendly);
- **Parallel rollouts** — the search is single-threaded today; arms are independent given the root, so near-linear speedup is available.

8 Reproducibility

- Engine and rules tests: `cargo test --release` (12 tests: piece set, legality, scoring, generator/validator consistency, bandit vs. greedy, budget compliance).
- Benchmarks: `cargo run --release --example selfplay`.
- Interactive play: `cargo run --release`; the think-time dial on the setup screen sets T per AI move. `BLOKUS_AUTO=<seconds>` launches a four-AI game directly.

Acknowledgments

Version 1 was co-authored with ShengHsiung Hsieh. The v2 Rust translation, the engine, the search, and this report were produced with Fable 5 (Claude, Anthropic).

References

- [1] P. Auer, N. Cesa-Bianchi, and P. Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256, 2002.
- [2] C. Browne et al. A survey of Monte Carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012.
- [3] E. J. Gumbel. *Statistical Theory of Extreme Values and Some Practical Applications*. U.S. National Bureau of Standards, 1954.

- [4] C. J. Maddison, D. Tarlow, and T. Minka. A* sampling. In *Advances in Neural Information Processing Systems 27*, 2014.
- [5] Mattel, Inc. *Blokus Instruction Sheet R1983*. https://service.mattel.com/instruction_sheets/R1983%20-%20Blokus.pdf